Student Attendance Management System (SAMS)

Project Proposal for the Academic Registrar's Department, Mbarara University of Science and Technology (MUST)

SWE2106 Internet Technologies and Web Design | Etivity 5
Team: [Team name] | Members: [Names and registration numbers] | Date: 10 November 2026

1. Problem Statement

Class attendance feeds directly into exam eligibility, yet it is often recorded on paper registers or passed-around sheets that the Registrar's office cannot easily consolidate. The challenges we propose to address, to be confirmed with the Registrar's office, are:

- **Time lost in class:** calling out names or circulating sheets takes lecture time.
- **Unreliable records:** paper registers get lost, damaged or filled in for absent students (proxy signing).
- **Slow consolidation:** totals per student must be counted by hand at the end of the semester.
- **Late discovery of problems:** low attendance is noticed only when exams approach, too late for advice or intervention.
- **Disputes over excused absences:** sick leave and other approved absences are poorly tracked, so students and staff disagree about the final percentage.
- **No audit trail:** it is hard to tell who changed an attendance entry, when, and why.

2. Users

User	Role in the system	Main needs
Lecturer	Creates class sessions and records attendance for their courses	Fast bulk marking; quick corrections within a set window
Student	Checks in to sessions, views own attendance, submits excuses	See own attendance % at any time; request excused absence
Head of Department (HoD)	Reviews excuse requests and monitors the department	Approve or reject excuses; see at-risk students
Registrar staff (Admin)	Manages students, courses, enrolment and policy	Consolidated reports and exam eligibility lists
University management (DVC)	Monitors attendance trends	Summary reports by department and course

3. Objectives

1. Replace paper registers with one digital attendance record per class session.
2. Cut the time spent taking attendance by allowing bulk marking and student self check-in with a session code.
3. Calculate each student's attendance percentage automatically per course.
4. Flag at-risk students early using a configurable threshold (default 75%, to be confirmed with the Registrar).

5. Track excused absences through a clear request and approval process.
6. Record every attendance change in an audit log.
7. Give the Registrar reliable exam eligibility lists at the end of the semester.

4. Scope

In scope

- User authentication and role-based access (Admin, Lecturer, HoD, Student).
- Management of students, courses and course enrolment.
- Creation of class sessions and attendance marking (present, late, absent, excused).
- Student self check-in using a time-limited session code.
- Excuse requests with HoD approval.
- Automatic attendance percentage, at-risk and exam eligibility reports.
- Express REST API, live database and a simple web frontend.
- Postman collection covering successful and invalid requests.

Out of scope for this phase

- Biometric, QR or GPS-based verification (possible future upgrade).
- Integration with existing university systems or SMS and email notifications.
- Examination results, transcripts and graduation clearance (future phases).
- Mobile app.

5. Proposed Solution

5.1 Core workflow

1. **Setup:** Admin registers students, courses and lecturers, and enrols students in courses.
2. **Session creation:** Lecturer creates a class session (course, date, time, venue). The system generates a short check-in code.
3. **Attendance:** Lecturer marks attendance in bulk for the enrolled list, or students check in with the code while the session is open.
4. **Correction:** Lecturer can edit entries within a set window (for example 48 hours), with a reason that is logged.
5. **Excuses:** Student submits an excuse for a missed session. HoD approves or rejects it, and approved absences become *excused*.
6. **Reporting:** Admin and HoD view attendance percentages, at-risk students and exam eligibility lists.

Every change is written to the audit log with the user, timestamp and action.

5.2 Key features

- Secure login with JWT and role-based permissions.
- Input validation (valid status values, valid student, course and session IDs, no duplicate entries for the same student and session, check-in only while a session is open).

- Automatic attendance percentage per student and course, with excused absences handled according to the configured policy.
- At-risk and exam eligibility reports based on a configurable threshold.
- Excuse request and approval workflow.
- Audit trail of all attendance changes.

5.3 Technology stack

Layer	Choice
Backend	Node.js with Express
Database	MongoDB Atlas (live cloud database) with Mongoose
Security	JWT authentication, bcrypt password hashing, environment variables for secrets
Validation	express-validator (or Joi)
Frontend	React (or HTML, CSS and JavaScript) consuming the API
Testing and docs	Postman collection with sample data
Design tools	Draw.io or Figma for diagrams and wireframes
Version control	GitHub with feature branches and pull requests

5.4 Planned API design

Method	Endpoint	Purpose	Access
POST	/api/auth/login	Log in and receive a token	Public
POST	/api/students	Register a student	Admin
POST	/api/courses	Create a course	Admin
POST	/api/courses/:id/enrol	Enrol students in a course	Admin
POST	/api/courses/:id/sessions	Create a class session	Lecturer, Admin
POST	/api/sessions/:id/attendance	Mark attendance in bulk	Lecturer
PUT	/api/attendance/:id	Correct an entry with a reason	Lecturer
POST	/api/sessions/:id/checkin	Student self check-in with code	Student
POST	/api/excuses	Submit an excuse request	Student
PATCH	/api/excuses/:id	Approve or reject an excuse	HoD
GET	/api/students/:id/attendance	View own attendance and percentage	Student, Admin
GET	/api/courses/:id/attendance-summary	Attendance summary for a course	Lecturer, HoD, Admin
GET	/api/reports/at-risk	Students below the threshold	HoD, Admin

Responses will use consistent JSON with appropriate HTTP status codes (200, 201, 400, 401, 403, 404, 409, 422), clear error messages and validation details.

6. Expected Benefits

- **More teaching time** because attendance is quick to record.
- **Trustworthy records** that are harder to falsify and easy to verify.
- **Early intervention** through at-risk reports during the semester, not at the end.

- **Fair handling of excused absences** with a documented approval trail.
- **Accountability** through a complete audit log of changes.
- **Less manual work** for the Registrar's office when preparing exam eligibility lists.

7. Team Roles and GitHub Workflow

Member	Responsibility
[Member 1]	Project lead, repository setup, authentication and roles
[Member 2]	Students, courses and enrolment API, database models
[Member 3]	Sessions, attendance marking and check-in logic, validation
[Member 4]	Excuses, reports and audit log, Postman collection
[Member 5]	Frontend, integration and deployment

Adjust the table to match your team size.

GitHub practice: the `main` branch is protected, each member works on a feature branch and merges through a pull request reviewed by at least one teammate, so every member has visible contributions. Commit messages follow a simple convention (feat, fix, docs).

8. Timeline

Date	Milestone
10 Nov 2026	Proposal submitted; repository created; roles agreed
10–11 Nov 2026	Express server, models, authentication and core endpoints; database connected
11 Nov 2026	Validation, attendance and excuse endpoints, frontend and Postman collection
12 Nov 2026	Final testing, GitHub pull requests merged, team presentation with live Postman demo

9. Testing Plan

The Postman collection will be organised by folder (Auth, Students, Courses, Sessions, Attendance, Excuses, Reports) with sample data and environment variables. Each endpoint will have:

- **Valid requests** that return the expected success response.
- **Invalid requests**, such as an invalid status value, a duplicate entry for the same student and session, a check-in with a wrong or expired code, a student not enrolled in the course, a student marking attendance, or a missing or expired token.

10. Risks and Mitigation

Risk	Mitigation
Short build time	Keep scope to the core attendance workflow; defer extras
Students checking in for absent friends	Time-limited code; lecturer can review and override with a logged reason
Unauthorised changes to records	Role-based access control, edit window and audit log
Merge conflicts	Feature branches, small pull requests, daily sync
Data loss or leaks	Managed cloud database, secrets kept in environment variables
Team members unable to explain the design	Shared design diagrams and a short walkthrough before the demo

11. Conclusion

The Student Attendance Management System gives the Academic Registrar's Department a practical, secure and extensible way to record attendance, spot at-risk students early and prepare exam eligibility lists with confidence. The team will deliver a working Express API, a live database and frontend, and a documented Postman collection by 12 November 2026, with room to extend into results and transcripts in later phases.